

CONCETTI CHIAVE SULL'IL0

TUTTE LE OPERAZIONI DI I/O FRA LE UNITÀ AVVENGONO SOSTANTIALMENTE ATTRAVERSO IL BUS. TALI OPERAZIONI COINVOLGONO I SEGUENTI PROBLEMI PRINCIPALI:

● CONTROLLO DEL BUS: QUALI UNITÀ USANO IN UN CERTO INTERVALLO DI TEMPO IL BUS PER LE LORO OPERAZIONI DI I/O, COME VENGONO GESTITE RICHIESTE IN "CONFLITTO" DA PARTE DI DIVERSE UNITÀ

- In ogni istante, una sola unità possiede il controllo del BUS, cioè decide quali operazioni di I/O eseguire
- Generalmente la CPU possiede il controllo del BUS (o dei BUS, se ve n'è più d'uno), ma può anche cedere temporaneamente questo ruolo ad altre unità

CONCETTO DI MASTER E SLAVE

- **MASTER**: CHI "DETTAGLIA" IL BUS IN UN DATO ISTANTE E DECIDE QUALI TRANSAZIONI (LETTURA O SCRITTURA) VANO EFFETTUATE, SU QUALI LINEE, E QUANDO. TIPICAMENTE È LA CPU, MA PUÒ CEDERE TEMPORANEAMENTE IL BUS AD ALTRI MASTER (DMA, CONTROLLER DI PERIFERICA)
- **SLAVE**: TUTTI GLI ALTRI DISPOSITIVI CHE RISPONDONO ALLE RICHIESTE DEL MASTER (MEMORIA, I/O)
- **ARBITRAGGIO**: QUANDO DUE MASTER VOGLIONO IL BUS CONTEMPORANEAMENTE, SERVE UN MECCANISMO DI ARBITRAGGIO CHE ASSEGNI LA PROPRIETÀ ESCLUSIVA DEL BUS AD UNO SOLO PER VOLTA
- IN CASO DI CESSAZIONE DEL RUOLO DI MASTER DA UN'UNITÀ FUNZIONALE AD UN'alTRA, O DI PIÙ RICHIESTE CONTEMPORANEE DEL BUS, OCCORRE UN MECCANISMO DI ARBITRAGGIO DEL BUS, CHE NE REGOLI L'UTILIZZO DA PARTE DELLE UNITÀ FUNZIONANTI.

ESISTONO DUE MECCANISMI PRINCIPALI DI ARBITRAGGIO: **CENTRALIZZATO E DISTRIBUITO**

→ ARBITRAGGIO CENTRALIZZATO

ARBITRO: UN'UNITÀ DESIGNATA (IL "BUS ARBITER") CHE RICEVE LE RICHIESTE DALLE PERIFERICHE E DECIDE A CHI DARE IL BUS

LINEE DI INTERCONNESSIONE: UNA LINEA DI RICHIESTA PER CIASCUN DEVIce. UNA LINEA DI GRANT CHE L'ARBITRO ASSEGNA PER IL MASTER SCELTO.

FUNZIONAMENTO: UN DISPOSITIVO TIRA IN BASSO LA SUA LINEA QUANDO VUOL'IL BUS. L'ARBITRO, SECONDO LA POLITICA ADOSSATA LA CORRISPONDENTE LINEA BUS GRANT. IL DISPOSITIVO CHE VEDE LA PROPRIA LINEA GRANT=O ASSUME IL RUOLO DI MASTER ED INIZIA LA TRANSIZIONE.

→ ARBITRAGGIO CENTRALIZZATO IN "Daisy Chain" (collegamento a "festone")



COLLEGAMENTO A CASATA: LE GRANT LINE PASSANO "IN SERIE" DA UN DEVIce ALL'ALTRO

PRIORITÀ IMPLICITA: IL PRIMO DEVIce IN CATENA HA LA MASSIMA PRIORITÀ, L'ULTIMO LA MINIMA. QUANDO L'ARBITRO ADOSSA BUS GRANT, ESSA ARRIVA PRIMA AL DEVIce 1, Poi

(SE 1 NON LA TRATTIENE) AL DEVIce 2, E COSÌ VIA

→ DAISS-CHAIN "MULTIFESTONE"



Festoni paralleli: si usano più "festoni" di request/grant, ognuno con una diversa gerarchia di priorità (es. festone 1: dispositivi {1,2,4}; festone 2: {3,5}).

Switch dinamico: l'arbitro può scegliere quale festone abilitare, cambiando così dinamicamente quali device hanno priorità più alta in quel momento.

Vantaggio: più flessibilità nel proteggere alcune periferiche da starvation.

- FESTONE 1 CHE COLLEGA LE UNITÀ 1, 2, 4
- FESTONE 2 CHE COLLEGA LE UNITÀ 3 E 5
- 2 LINEE DI BUS REQUEST, UNA OGNIUNA
- L'ARBITRO PUÒ CAMBIARE DINAMICAMENTE LA PRIORITÀ DEI 2 FESTONI

→ BUS ACKNOWLEDGE ("BUS ACK")

Problema residuo: nei modelli appena visti, il master non "rilascia" esplicitamente il bus: l'arbitro deve aspettare un timeout o vigilare su una signal spur ("grant" torna alta).

Soluzione: aggiungere una terza linea BUS Ack (attiva-low).

1. Master richiede e ottiene grant.

2. Subito dopo aver preso il bus, il master tira in basso Bus Ack → segnala all'arbitro "ho iniziato davvero la mia transazione, ora puoi preparare il prossimo grant."

3. Quando il master ha finito, riporta alta Bus Ack → l'arbitro scopre subito che il bus è libero e concede subito al successivo.

Diagramma temporale: si vede come la seconda periferica, già in pending request, riceva grant immediatamente dopo che la prima rilascia via Ack, senza dover aspettare di nuovo l'intervento dell'arbitro sul grant line.

➤ Nei meccanismi di arbitrato descritti, le unità non sono in grado di sapere quando un'unità che aveva chiesto e ottenuto il controllo del BUS lo "rilascia"

➤ Si migliora l'efficienza dell'arbitraggio aggiungendo una terza linea di controllo: il segnale di "accettazione": BUS acknowledge (Bus Ack)

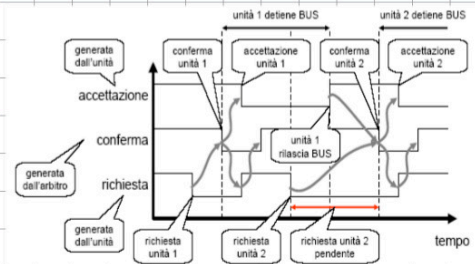
➤ La linea di accettazione viene attivata dalle unità, non dall'arbitro:

- ✓ un'unità richiede e ottiene dall'arbitro il controllo del BUS
- ✓ non appena l'arbitro le invia la conferma, l'unità che prende il controllo del BUS attiva la linea di accettazione (Bus Ack) e disattiva la linea di richiesta
- ✓ non appena l'arbitro vede attivarsi la linea di accettazione, disattiva la linea di conferma

➤ Così, subito dopo che l'unità ha preso il controllo del BUS, le linee di richiesta e di conferma sono già libere per un'altra unità può dunque inviare subito una nuova richiesta, che resterà "pendente";

➤ Questa seconda unità riceverà la conferma non appena la prima unità avrà "rilasciato" il controllo del BUS, disattivando la linea di accettazione (Bus Ack)

➤ Il meccanismo di accettazione è più complesso, ma è più efficiente



→ ARBITRAGGIO DISTRIBUITO



➤ Il meccanismo di arbitrato distribuito a "n" linee prevede una linea di richiesta BUS per ogni unità. Non esiste alcun arbitro del BUS

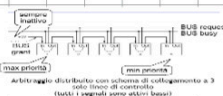
➤ Le unità hanno priorità diverse e fissate. Le unità osservano tutte le linee di richiesta BUS

➤ Ogni unità può attivare solo la propria linea di richiesta BUS

➤ Quando l'unità deve diventare MASTER, si accerta che nessun'altra unità a priorità superiore alla sua abbia attivato la propria linea di richiesta BUS

➤ Se così è, l'unità attiva la sua linea di richiesta BUS e ottiene il controllo del BUS, diventando MASTER

➤ Confronto: si risparmia il costo dell'arbitro, ma il numero di linee di controllo cresce con quello delle unità



➤ Si può migliorare l'arbitraggio distribuito a n linee usando solo 3 linee di controllo:

• BUS occupato (BUS busy): questa linea viene mantenuta attiva dall'unità che detiene correntemente il controllo del BUS

• conferma BUS (BUS grant): è la linea di conferma, collegata a festone

• richiesta BUS (BUS request): l'unità attiva questa linea per richiedere il BUS

● FUNZIONAMENTO E TEMPORIZZAZIONE DEL BUS

• Le attività del calcolatore si sviluppano per cicli di BUS: in ogni ciclo avviene un'operazione (o transazione)

• Le operazioni sono governate dal MASTER che detiene il controllo del BUS

• Gli SLAVE non possono dare inizio a un'operazione in modo autonomo

• Operazioni: trasferimento di dati tra MASTER e SLAVE

• Operazione di lettura: un dato viene trasferito da uno SLAVE al MASTER

• Operazione di scrittura: un dato viene trasferito dal MASTER a uno SLAVE

• Nota bene: il punto di vista per stabilire la direzione del trasferimento (lettura o scrittura) è sempre e solo quello del MASTER che detiene (in quel momento) il controllo del BUS

• La temporizzazione è il modo con cui gli eventi (lettura, scrittura, ecc.) sono coordinati sul BUS

• Temporizzazione Sincrona

- Eventi determinati da un "clock". Bus cycle=Clock Cycle
- Tutto sincronizzato sui "fronti" del clock
- Molti "eventi" occupano uno o più cicli di clock

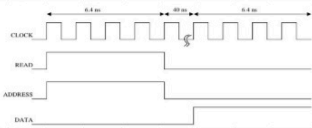
• Temporizzazione Asincrona

- L'occorrenza di un evento dipende da quella dell'evento precedente

• Modalità sincrona è più semplice ma meno flessibile: comunque la più usata.

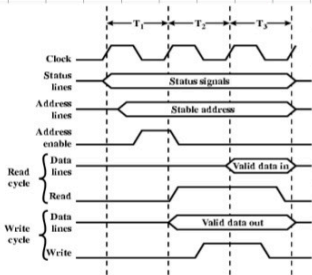
• Modalità asincrona consente a dispositivi di diversa "velocità" di condividere efficacemente il BUS

→ ESEMPIO DI "LETURA" SINCRONA SEMPLIFICATA



Clock a 625 MHz
 Ogni trasmissione sul bus impiega 4 cicli di clock = 6.4 ns
 Tempo di ciclo della memoria (tempo che ipotizziamo serva alla memoria per posizionare sul bus i dati richiesti dalla lettura): 40 ns
 Tempo di lettura di una parola da 32 bit su bus da 32 bit: 52.8 ns
 • Stiamo assumendo che ogni trasmissione di segnali di controllo (Read), indirizzi, e dati impieghi 4 cicli di clock per essere trasmessa sul bus.

→ ESEMPIO DI TEMPORIZZAZIONE SINCRONA



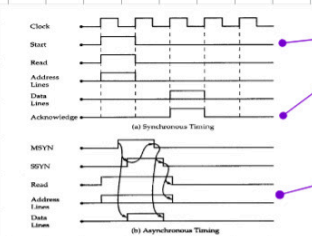
- Questo esempio serve ad illustrare dei possibili segnali che vengono scambiati durante delle operazioni di lettura/scrittura in modalità sincrona
- E' ovviamente solo un esempio per scopi didattici
- Diversi bus impiegano diversi segnali, con diversi nomi e possono avere modalità di scambio segnali diverse
- Ma resta sempre vero che tutto è «temporizzato» dal bus nella modalità sincrona
- «pendenza» fronte di salita/discesa del clock non ha importanza per la trattazione «logica» di questo corso
- Durante il primo ciclo di clock il master (ad es. la CPU) pone sul bus gli indirizzi (ad es. di una locazione di memoria da leggere). Al tempo stesso potrebbe inviare dei segnali di stato (non di interesse in questo esempio)
- In questo esempio il Segnale di «address enable» serve a indicare allo slave che gli indirizzi sono stabili e li può leggere
- Sul secondo ciclo di clock può avvenire READ o WRITE
- Per WRITE prima bisogna metterci sul bus i dati da scrivere
- Nel caso del READ otteniamo dati sul bus dopo un ritardo di un ciclo (il tempo che prima avevamo ipotizzato pari al tempo di ciclo della memoria)

→ ESEMPIO DI "LETURA" ASINCRONA



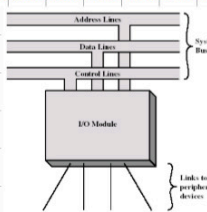
- Segnali di Read e Acknowledge attivi «bassi»
- Lo Slave (ad es. memoria) segnala con «Acknowledge» al master che i dati che voleva leggere sono disponibili sul bus
- Segnali di Read e Acknowledge attivi «bassi»
- Slave (ad es. memoria) segnala con «Acknowledge» al master di avere «copiato» i dati che master voleva «scrivere»

→ ESEMPIO DI TEMPORIZZAZIONE SINCRONA ED ASINCRONA



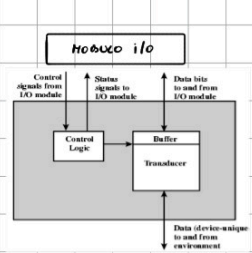
- N.B. anche nel caso sincrono possiamo avere dei segnali di controllo di «sincronismo».
- Serve a rendere maggiormente sicura la comunicazione, anche in presenza di «errori» di sincronismo sul clock
- MSYN: segnale di sincronismo del Master
- SSYN: segnale di sincronismo dello Slave
- Master manda READ e indirizzi e segnala operazione a slave con segnale controllo MSYN
- Slave mette dati richiesti in lettura sul bus e fornisce «acknowledge» con segnale SSYN

• INTRODUZIONE ALLE MODALITÀ DI I/O



- Unità di Insieme di di I/O/O = Moduli
- Ciascuna modulo di I/O copre due funzioni fondamentali:
 - controlla una o più periferiche connesse al sistema
 - comunica dati da/alla periferica a/dagli altri moduli dell'architettura
- Perché le periferiche non vengono connesse direttamente al bus di sistema?
- Esiste una grande varietà di periferiche
 - in termini di funzionalità
 - stampante, modem, etc.
 - in termini di velocità di trasferimento
 - Da decine di bit/sec (tastiera) a Gigabit connessione rete
 - in termini di formato e lunghezza dei dati
 - bisognerebbe adottare una logica diversa per ogni formato
- E' dunque necessario un modulo che faccia da «interfaccia» tra CPU, memoria e periferiche

SCHEMA FUNZIONALE DI UNA PERIFERICA



LO SCHEMA "FUNZIONALE" DI UNA PERIFERICA (IL BLOCCO CHIAMATO "MODULO I/O") METTE IN EVIDENZA 3 REE DI SEGNALE E 3 BLOCCHI INTERNI:

→ 1) Le linee di segnale: tra CPU (I/O module) e periferica

- Control signals (READ/WRITE)
 - Vengono generate dal modulo I/O per dire al Control Logic della periferica se deve eseguire un'operazione di lettura (READ) o scrittura (WRITE).
- Data bits
 - Sono i bit «veri» che percorrono il bus tra modulo I/O e Buffer della periferica: per esempio il codice ASCII di un carattere premuto su tastiera, oppure il dato letto da un sensore.
- Status signals (es. READY/NOT READY)
 - La periferica informa il modulo I/O sul suo stato interno: «sono pronta a un'altra operazione?», «ho finito di trasmettere dati?», «ho incontrato un errore?».

→ 2) I tre blocchi interni alla periferica

1. Control Logic: È la «mente» che interpreta i control signals e coordina il trasferimento dati.
2. Buffer: Un'area di memoria «tamponata» (FIFO o registro singolo) che separa i due domini temporali: il bus (che può essere veloce e trasferire 32 bit in burst) il dispositivo vero e proprio (magari lento, come un disco o un convertitore analogico). Permette di accumulare dati in arrivo (write) o di consegnarli a ritmo costante (read).

→ 3) Transducer: Il «trasduttore» è l'interfaccia elettrico/fisico tra il Buffer e il mondo esterno:

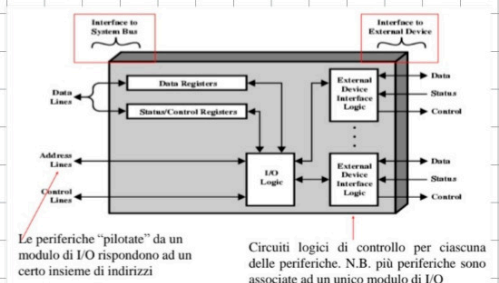
- Per una tastiera, trasforma la pressione di un tasto in un codice elettronico (ASCII);
- Per un sensore di pressione, converte una carica elettrica in un numero digitale;
- Per un motore passo-passo, riceve bit di controllo e genera le tensioni ai coil.

Flusso di un'operazione di lettura (READ)

1. CPU (modulo I/O) tira READ low e imposta l'indirizzo periferica.
2. Control Logic periferica riceve il segnale, estrae dal Buffer il prossimo dato.
3. I bit passano dal Buffer al bus come Data bits.
4. Una volta che il Buffer è vuoto, la Control Logic abbassa lo Status=NOT READY per dire «non ho altri dati» (oppure «sono busy» finché non ne arrivano di nuovi).

Flusso di un'operazione di scrittura (WRITE)

1. CPU invia WRITE + Data bits sul bus.
2. Control Logic li preleva e li mette nel Buffer.
3. Quando il Buffer raggiunge una certa soglia, la Control Logic porta Status=READY per dire «ho dati pronti per il Transducer».
4. Il Transducer li preleva dal Buffer e li converte in comandi fisici verso l'unità meccanica/elettrica esterna.



Le periferiche «pilotate» da un modulo di I/O rispondono ad un certo insieme di indirizzi

Circuiti logici di controllo per ciascuna delle periferiche. N.B. più periferiche sono associate ad un unico modulo di I/O

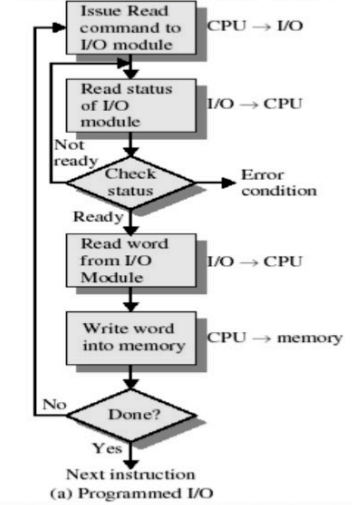
Modalità di I/O

	No Interrupts	Use of Interrupts
I/O-to-memory transfer through processor	Programmed I/O	Interrupt-driven I/O
Direct I/O-to-memory transfer		Direct memory access (DMA)

I/O programmato

- La responsabilità delle comunicazioni I/O e del trasferimento dati è interamente del "programmatore", ovvero della CPU
- Il processore invia/riceve dati direttamente dal modulo I/O, e controlla periodicamente lo stato della periferica
- La velocità di trasferimento dipende dalla periferica
- Il processore attende che la periferica completi il suo compito prima di proseguire l'esecuzione del programma

ESEMPIO DI FLOW-CHART I/O PROGRAMMATO ("READ")



1) I/O PROGRAMMATO

CHI CONTROLLA: LA CPU, IN MODO ESPLICITO E CONTINUO

COME FUNZIONA:

- IL PROGRAMMA DELL'APPLICAZIONE ESEGUE ESPlicitAMENTE ISTRUZIONI DI I/O
- DOPO OGNI COMANDO, LA CPU ATTENDE (POLLING) IL COMPLETAMENTO DELL'OPERAZIONE LEGGENDO RIPETUTAMENTE UNO STATUS BIT PRODOTTO DALLA PERIFERICA
- QUANDO IL DEVICE È PRONTO, LA CPU CONTINUA L'ISTRUZIONE SUCCESSIVA

2) I/O PILOTATO DA INTERRUZIONI

CHI CONTROLLA: CPU + CONTROLLER DELLE INTERRUZIONI + INTERRUPT SERVICE ROUTINES (ISR)

COME FUNZIONA:

- LA CPU INVIA ALLA PERIFERICA IL COMANDO DI LETTURA/SCRITTURA (SENZA ATTENDERE)
- PROSEGUE L'ESECUZIONE DEL PROGRAMMA SENZA POLLING
- QUANDO IL DEVICE HA TERMINATO, GENERA UN SEGNALE DI INTERRUPT
- IL CONTROLLER DI INTERRUZIONI "FERMA" LA CPU, SALVA IL CONTESTO, E SALTA ALLA ISR ASSOCIATA
- NELL'ISR SI LEGGE O SCRIVE IL DATO (E SI RIPRISTINA LO STATO)
- AL RITORNO DALL'ISR, LA CPU RIPRENDE IL PROGRAMMA PRINCIPALE DA DOVE ERA STATA INTERRUPTA

3) DMA (DIRECT MEMORY ACCESS)

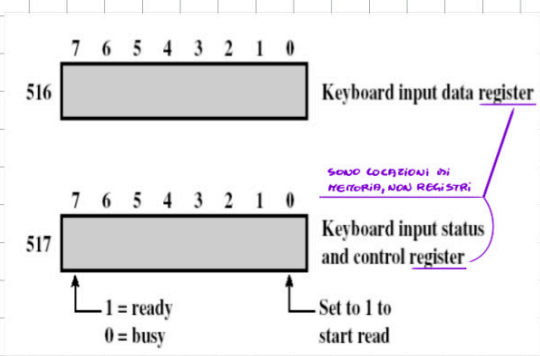
CHI CONTROLLA: UN CONTROLLER DMA DEDICATO, DISTINTO DALLA CPU

COME FUNZIONA:

- LA CPU CONFIGURA IL DMA (INDIRIZZO SORGENTE PERIFERICA, INDIRIZZO DESTINAZIONE MEMORIA, DIMENSIONE DEL BLOCCO) E LO AVVIA
- IL DMA PRENDE POSSESSO DEL BUS DI MEMORIA, TRASFERISCE AUTONOMAMENTE BLOCCHI DI DATI TRA PERIFERICA E RAM, SENZA GINVOLGERE LA CPU A PAROLA
- SOLO A COMPLETAMENTO DEL BLOCCO, IL DMA GENERA UN INTERRUPT ALLA CPU PER SEGNALARE LA FINE DEL TRASFERIMENTO

I/O PROGRAMMATO: LE ISTRUZIONI ALLA PERIFERICA

- Le istruzioni abitualmente usate per il trasferimento dati dai registri alla memoria vengono facilmente utilizzate come istruzioni I/O
- Per identificare in modo univoco la periferica, le si associa un indirizzo
- In tal modo è possibile "mapparla" in memoria associandole uno dei possibili indirizzi della linea indirizzi del bus ("memory-mapped" I/O)
- Altrimenti, si può modificare il formato dell'istruzione in modo da distinguere indirizzi di memoria e indirizzi di periferica ("isolated" I/O)



ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load AC	"1"	Load accumulator
	Store AC	517	Initiate keyboard read
202	Load AC	517	Get status byte
	Branch if Sign = 0	202	Loop until ready
	Load AC	516	Load data byte

(a) Memory-mapped I/O

ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load I/O	5	Initiate keyboard read
201	Test I/O	5	Check for completion
	Branch Not Ready	201	Loop until complete
	In	5	Load data byte

(b) Isolated I/O

1) MEMORY-MAPPED I/O

PRINCIPIO: GLI INDIRIZZI USATI PER ACCEDERE A MEMORIA "REALI" (RAM) SONO CONDIVISI CON QUELLI DELLE PERIFERICHE

COME FUNZIONA:

- ALLA PERIFERICA (ES. TASTIERA) SONO ASSEGNATI ALCUNI INDIRIZZI DI MEMORIA IN AZITO NELLO SPAZIO DEGLI INDIRIZZI (QUI 516 E 517)
- PER AVVIARE LA LETTURA SI SCRIVE UN VALORE NELLO "STATUS/CONTROL REGISTER" ALL'INDIRIZZO 517. (STORE AC 517)
- QUANDO LA PERIFERICA È PRONTA (STATUS=1), SI FA UN SEMPLICE LOAD AC 516 PER PRELEVARE IL DATO (DATA REGISTER) DA QUELL'INDIRIZZO
- NEL FRATEMPTO LA CPU PUÒ TRATTARE LA PERIFERICA COME UN NORMALE LOCAZIONE DI MEMORIA: LOOP DI POLLING SULLO STATUS (LOAD AC 517; "BRANCH IF SIGN" 202 FINO A CHE NON DIVENTA 1), POI LOAD DATI (516)

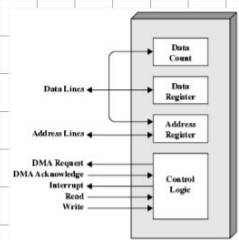
2) ISOLATED (o PORT-MAPPED) I/O

PRINCIPIO: I DEVICE HANNO UNO "SPAZIO INDIRIZZI SEPARATO", ACCESSIBILE SOLO CON ISTRUZIONI DEDICATE DI I/O

COME FUNZIONA:

- ISTRUZIONE SPECIALE LOAD I/O 5 (ADDRESS 200) INVIA AL CONTROLLER TASTIERA UN COMANDO DI "START READ"
- Test I/O 5 legge un registro di status interno: se non pronto, fa "Branch Not Ready 201" per rimanere in ciclo.
- Quando il device risponde "pronto", si usa IN 5 (lettura del dato) per trasferirlo nell'accumulatore.

DMA → DIRECT MEMORY ACCESS



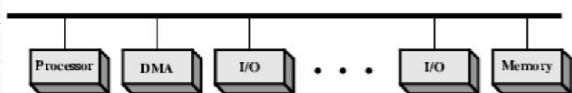
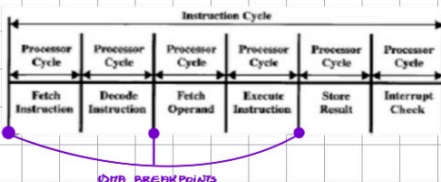
- Limiti del I/O programmato e del I/O pilotato con interruzioni
 - la velocità di trasferimento è limitata dalla velocità con cui il processore può "verificare" e "servire" una periferica
 - le istruzioni processore di I/O sono comunque eseguite dal processore
- Il modulo per l'Accesso Diretto alla Memoria (DMA) supera tali limiti, soprattutto quando il numero di trasferimenti è elevato
- E' un modulo hardware aggiuntivo al modulo I/O
 - la locazione iniziale su cui trasferire dati
 - il numero di parole da trasferire
 - l'indirizzo relativo modulo I/O del di
- Richiede in input:
 - la locazione iniziale su cui trasferire dati
 - il numero di parole da trasferire
 - l'indirizzo relativo modulo I/O del di

ALGORITMO BASE CON IL DMA

- Il DMA riceve dalla CPU le informazioni necessarie per il trasferimento
- Il processore prosegue il programma in corso nell'esecuzione del
- Nel frattempo il DMA effettua il trasferimento richiesto
- Al termine del trasferimento il DMA emette un'interruzione per segnalare al processore che l'operazione di I/O è conclusa

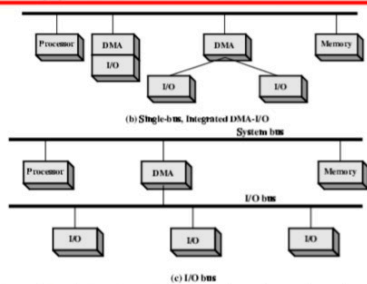
MODALITÀ DI TRASFERIMENTO CON DMA

- Block transfer — la modalità più veloce
 - il DMA diventa "master" del bus per il trasferimento dei dati
 - il processore ne riprende il controllo
 - utile per unità I/O del tipo disco o stampanti
- Transparente solo a
 - la modalità più lenta
 - il DMA effettua un trasferimento solo quando il bus non serve al processore

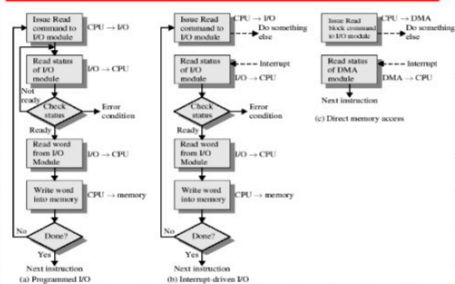


- Il DMA funziona da "secondo processore"
- Ogni trasferimento richiede due cicli di clock

Configurazioni più efficienti del modulo DMA (2)



I/O programmato, I/O con interruzioni e DMA a confronto



Evoluzione della gestione del I/O

- Controllo totale da parte della CPU - I/O programmato
- I/O con interruzioni
- I/O con DMA
- Canali I/O: il modulo di I/O viene dotato di un processore in grado di eseguire un proprio set di istruzioni I/O senza l'intervento della CPU
- Processori I/O: memoria locale il canale I/O viene dotato di
 - molto usati nel controllo di terminali interattivi